

Semantically Indexing Messy and Decompiled Codebases

June 21, 2026 • 36 Sources

EXECUTIVE SUMMARY

Semantic search over a decompiled codebase fails for a specific, well-documented reason: mainstream code embedding models lean heavily on identifier names to encode meaning, and decompilation destroys exactly that signal, replacing developer names with placeholders such as `v1`, `var2`, `sub_401000`, or Lua register slots [5][24][25]. A query like "where is the player respawn logic" embeds close to source that says "respawn," but a decompiled function named `function_3` that implements respawning carries almost no lexical hint, so a naive vector index returns noise. Empirical work confirms the sensitivity: `code2vec` accuracy collapses under trivial identifier perturbation [25], and renaming-robustness is now treated as a first-class research problem [23][24][26].

The strongest evidence points to a layered solution rather than a single model. The highest-leverage move for messy code is to **stop embedding the raw decompiled text and instead embed a semantic surrogate** — a natural-language summary or a name-recovered version produced by an LLM. Two converging literatures support this: (1) decompiler symbol-recovery models (DIRTY recovers original developer names 66.4% of the time [1]; VarBERT reaches ~54% on optimized binaries [2]; ReSym, LmPa, and SymGen push name/type recovery further using LLMs plus program analysis [3][4][6]); and (2) repository summarization pipelines that generate bottom-up natural-language descriptions of functions and files, then embed those descriptions, explicitly to bridge the gap between natural-language queries and opaque code [20][21][22].

Around that core, four supporting techniques materially raise retrieval quality: AST-based chunking that keeps functions intact and prepends structural context before embedding [12][13][14]; naming-agnostic / structural representations that learn from control flow and data flow rather than identifiers [23][24][26][27][28]; a hybrid retrieval architecture combining BM25 lexical search with dense vectors via Reciprocal Rank Fusion plus a cross-encoder reranker [30][31][32]; and a current code-specialized embedding model such as `voyage-code-3` [9]. Production code-search systems (Sourcegraph, Cursor, Turbo) independently converge on the same hybrid, structure-aware, incrementally-indexed pattern [15][16][17][18][19].

This report recommends a concrete pipeline for a decompiled (e.g., Lua) codebase: improve decompiler output first [33], recover/normalize names with an LLM pass, generate hierarchical NL

summaries, embed the *enriched* text with a code model, and serve queries through hybrid retrieval with reranking.

INTRODUCTION

The question driving this research is practical: what is the best way to build a *semantic* index over a codebase whose identifiers carry little or no meaning — the canonical case being decompiled code. The request explicitly asked to investigate decompilation-specific techniques first, and to fall back to general semantic-indexing best practices if the specific techniques are thin. As it turns out, both literatures are rich, and the best answer combines them.

Scope. This report covers (a) why standard semantic code indexing degrades on identifier-poor code; (b) decompilation-specific remedies — neural and LLM-based variable/function name and type recovery; (c) the summarize-then-embed strategy; (d) structure-aware and naming-agnostic representations; (e) the general-purpose retrieval architecture (chunking, embedding model choice, hybrid search, reranking); and (f) an end-to-end recommended pipeline, including notes specific to Roblox/Luau bytecode decompilation. It excludes training new foundation models from scratch and binary lifting internals beyond what affects index quality.

Why decompiled code is the hard case. Compilation is lossy. As the reverse-engineering survey states plainly, decompiled code "is notoriously imprecise, often containing type inaccuracies, goto statements for control flow, and meaningless variable names, such as var1 and var2" [5]. The same loss occurs in Luau: the bytecode compiler discards "variable names (except globals/constants), comments, and formatting," so "variable names become register numbers, control flow becomes jumps, and elegant expressions become sequences of stack operations" [33]. Whatever indexing strategy works for clean source will systematically underperform here unless it is adapted.

Methodology (summary). This is a deep-mode run: parallel web retrieval across eight angles, evidence triangulated across 36 registered sources (peer-reviewed papers from USENIX Security, IEEE S&P, ISSTA, EMNLP, ACL/CCS; vendor engineering documentation; and practitioner write-ups), with claims cited inline. The full methodology, source registry, and evidence ledger are in the appendix and accompanying `sources.jsonl` / `evidence.jsonl` files.

Assumptions (high-materiality, surfaced explicitly). (1) The target is read-mostly retrieval/search and LLM-assisted code understanding, not recompilation. (2) The decompiled language is high-level enough to parse into an AST (true for Luau and for C-like decompiler output), which enables structural techniques. (3) An LLM is available at index time for a one-off enrichment pass — the cost model assumes offline batch indexing with cheap incremental updates, matching how Cursor and Turso operate [16][17]. (4) Vendor leaderboard claims are treated as lower-credibility marketing unless corroborated; this is flagged where it matters [11].

MAIN ANALYSIS

The six findings below move from diagnosis to architecture: why naive indexing fails (Finding 1), the three orthogonal ways to restore semantic signal — repair, replace, represent (Findings 2–4), the retrieval architecture that serves it (Finding 5), and an end-to-end pipeline validated against production systems (Finding 6).

Finding 1: Naive semantic indexing fails on decompiled code because embeddings depend on identifier names

The first thing to establish is *why* the obvious approach — chunk the decompiled files, embed each chunk with an off-the-shelf code embedding model, and do nearest-neighbor search — produces poor results. The root cause is that code representation models are disproportionately sensitive to identifier names, and decompilation is precisely the transformation that strips meaningful identifiers.

The clearest evidence comes from robustness studies of code2vec, a widely used code-embedding model. A controlled study from the University of Waikato found that code2vec is "very sensitive to variable names," and that "even minor changes in variable names (e.g., typos or alternative naming) significantly affect the model's" predictions; the model's robustness "is noticeably compromised" when confronted with non-human naming conventions such as machine-generated code [25]. This is not a code2vec-specific quirk. A follow-on study reframed the same finding as a regularization problem: deliberately obfuscating variable names during training "produces an embedding model that is both impervious to variable naming and more accurately reflects code semantics," because hiding informative names "will force the model to learn" from structure instead [24]. The implication runs in reverse for our setting: if models lean on names, then removing names — as decompilation does — removes the dominant feature the model was relying on.

The reverse-engineering literature describes the downstream consequence in concrete terms. Stripped binaries "use address-based identifiers (e.g., `sub_401000`) for functions, which provide little information about their behavior and make analysis difficult" [4]. Decompiled output inherits placeholder names like `var1/var2` that "provide little information about their behavior" [4][5]. When such tokens dominate a chunk, a dense embedding of that chunk lands far from the natural-language query that describes what the code *does*, because the lexical bridge the model expects is absent. This is the same "semantic gap between natural language ... and code" that the bug-localization literature identifies as the central obstacle for LLM-based code retrieval [20].

There is a second, subtler failure mode: even when identifiers are present but *inconsistent*, retrieval suffers. The naming-agnostic code search work (NACS) motivates its entire approach by "the challenge brought by the naming issue" — different code with the same meaning uses different names, and models that bind meaning to specific identifier strings cannot align them [23]. Decompiler output is the extreme of this: two functions that were identical in source can decompile to differently-numbered registers and temporaries, so a name-dependent index treats semantically identical code as unrelated.

Taken together, these sources establish the design constraint for everything that follows: an index for decompiled code must derive its semantic signal from something *other than* raw identifiers — either by reconstructing meaningful names before indexing (Finding 2), by replacing the code with a natural-language surrogate (Finding 3), or by representing structure directly (Finding 4). Three independent research threads — code2vec robustness [24][25], naming-agnostic search [23], and stripped-binary analysis [4][5] — converge on this same conclusion, which is why it is stated with high confidence.

Finding 2: Recover meaningful names and types before indexing (the decompilation-specific layer)

Because the core problem is missing identifiers, the most direct decompilation-specific remedy is to *recover* them. This is a mature and active research area, which directly answers the user's first question — there is substantial decompilation-specific work, not just a generic fallback.

The foundational neural approaches recover names and types as a post-processing step on decompiler output. **DIRTY** (Decompiled variable ReTYper) is a Transformer model trained on ~1 million functions and 368 million tokens of C scraped from GitHub; it "postprocesses decompiled files, recommending variable types and names given their context" and recovers "the original names written by developers 66.4% of the time and the original types 75.8% of the time" [1]. **DIRE**, its predecessor, combined a lexical and an AST-based structural encoder for name recovery [1]. **VarBERT** improved generalization by pre-training a BERT model on 5.2 million human source functions with masked and *constrained* masked language modeling (masking only variable tokens), then fine-tuning on IDA/Ghidra decompiler output; it predicts developer names with ~54% accuracy on O2-optimized binaries — "strictly better" than DIRE (~36%) and DIRTY (~38%) on a common subset [2]. Crucially, VarBERT's design is decompiler-agnostic: it "can be adapted for use with any other decompiler capable of generating C-Style decompilation output" [2], which means the same recipe transfers to other lossy front-ends.

The current generation uses LLMs, often combined with program analysis, and is more powerful for our purpose because it can name *functions* (not just locals) and propagate names across the call graph.

ReSym "combines fine-tuned LLMs with Prolog-based reasoning to recover variable and data structure symbols from stripped binaries" [3][6], integrating LLM predictions with constraint-based program analysis to improve accuracy [4]. **LmPa** is especially relevant: it "divides the task into many LLM queries and uses program analysis to correlate and propagate the query results," so a meaningful name derived in one query window "can be propagated to other places in the program following strict program semantics," and "a newly generated callee function name is propagated to the invocation sites," enriching the context available to later queries in an iterative loop [3]. This name-propagation idea matters because decompiled functions are individually opaque but collectively constrained — a well-named callee disambiguates its callers. **SymGen** similarly fine-tunes models to recover function names from decompiled code, and the REBench benchmark now evaluates this whole class of "name recovery and type inference via decompiled code" systematically, noting that decompiled code "retains richer semantic information than assembly," which is why it is the preferred input representation [6].

A complementary path is to improve the decompiled code itself before any naming pass.

LLM4Decompile is the first large open-source LLM series (1.3B–33B) trained for decompilation; its refinement variant fine-tunes on Ghidra output and improves readability and re-executability substantially over baseline tools [7]. The 2026 paradigm survey documents a two-phase trend ("skeleton to skin"): first recover structure, then generate "human-readable source code with meaningful identifiers" [8]. **DeGPT** specifically targets readability "through comment generation and variable renaming" [8]. For an indexing pipeline, the takeaway is that an LLM "renaming agent to assign descriptive, human-readable names to decompiled functions" [4] can be run as a batch enrichment step, after which conventional code-search indexing becomes far more effective.

Two cautions temper this layer. First, accuracy is bounded and optimization-dependent: recovery rates of 54–66% mean a meaningful fraction of names remain wrong or generic, and performance "exhibits a strong dependency on the quality" of the underlying decompiler output [4]. Second, these models were largely built and benchmarked for C/C++ from IDA/Ghidra [1][2][6]; applying them to Lua requires either using the language-agnostic LLM-renaming approach [4] or treating the transfer-learning recipe [2] as a template rather than a drop-in. The robust, portable version of this finding is therefore: **run an LLM renaming/retyping pass over decompiled functions, propagating names across the call graph, as a pre-index enrichment stage** — a technique that is well supported across DIRTY, VarBERT, ReSym, LmPa, and SymGen [1][2][3][4][6].

Finding 3: Embed a natural-language surrogate (summarize-then-embed) — the highest-leverage move for messy code

If name recovery repairs the input, summarize-then-embed sidesteps the problem entirely: instead of embedding identifier-poor code, you embed an LLM-generated natural-language description of what the code does, and index that. This is the single most practical and broadly supported technique for messy code, because it converts an identifier-matching problem into a natural-language matching problem — the regime where embedding models are strongest.

The rationale is stated directly in the bug-localization literature, which reframes code retrieval "as a natural language reasoning task by transforming codebases into hierarchical NL summaries" precisely to overcome "the semantic gap between natural language bug reports and code" [20]. The construction is bottom-up and hierarchical. For each function, an LLM is prompted with its code (and surrounding context) to produce a concise summary; file-level summaries aggregate function summaries; directory- and repository-level summaries aggregate those [20][22]. RepoSummary formalizes the method: it summarizes "at the method-level and then file-level," partitioning each method summary into "description ... workflow ... and quality," then vectorizes the descriptions with Sentence-BERT to build a semantic-similarity matrix over functions [21]. The hierarchical local-LLM study confirms the practical motivation — "local LLMs may struggle with summarizing entire files or packages, [but] they are effective at summarizing smaller units, such as methods or functions" — so the codebase is decomposed via an AST parser into units, summarized, and aggregated upward [22].

Why this is especially powerful for decompiled code: a summary is robust to the exact failure mode of Finding 1. Whether a function is named `respawnPlayer` or `function_3`, an LLM reading its body can often infer that it "resets the character model and teleports the player to a spawn point," and *that sentence* embeds close to the query "player respawn logic." The summary launders away the meaningless identifiers and emits the semantic content that the query targets. This is the same mechanism the reverse-engineering survey lists among the dominant LLM uses for decompiled code — "restating logic ... and generating summaries" [5] — applied at index time rather than read time.

A key implementation detail repeatedly emphasized is **context enrichment before embedding**. Summaries (and code chunks) should be prepended with metadata: file paths, the scope chain (e.g., `Class > Method`), dependency lists, and the repository-level "seed context" that tells a low-level summary its architectural role [13][20]. The code-chunk engineering write-up makes the point sharply: "Embedding models are trained on natural language. When you embed `async getUser(id: string)`, the model doesn't inherently know this is inside a `UserService` class or that it uses a `Database`. By prepending this context, the embedding captures semantic relationships that pure code misses" [13]. For decompiled code, this enrichment is doubly valuable because the surrounding structure (call graph position, which globals or Roblox APIs are referenced) often carries more reliable semantic signal than the function body's local names.

The trade-offs are cost and fidelity. Summarizing every function with an LLM is an offline batch expense, and summaries can hallucinate or omit detail. The mitigations are: index incrementally so summaries are regenerated only for changed units [17]; keep both the summary and the raw code chunk in the index (embed the summary for recall, return the code for the LLM to read); and use the cheapest adequate model for summarization, reserving stronger models for renaming. Across three independent academic pipelines [20][21][22] plus production engineering guidance [13], the convergent recommendation is to **generate hierarchical NL summaries, enrich them with structural metadata, and embed the summaries as the primary semantic index** — with raw code retained as the payload.

Finding 4: Represent structure, not just text — AST chunking and naming-agnostic embeddings

The third route to identifier-independence is to capture the *structure* of code — control flow, data flow, AST shape, call graph — which survives decompilation even when names do not. This operates at two levels: how you chunk, and what you embed.

Chunk on syntax, not on line count. The cAST work shows that naive fixed-size or line-based chunking "often splits meaningful units (e.g., functions and classes) across different chunks, losing structural integrity and context" [12]. cAST parses code into an AST (via Tree-sitter) and applies a recursive "split-then-merge" algorithm: traverse top-down to keep large nodes whole, split nodes that exceed the size budget, then greedily merge small adjacent siblings to maximize per-chunk information density [12]. The measured effect is consistent gains across retrievers — roughly "1–3 points in Precision and ... Recall" improvements on code RAG [12]. This matters for decompiled code because keeping a whole

reconstructed function (or a tight cluster of related ones) in a single chunk preserves the control-flow context that gives the function meaning, which is exactly what a name-poor function needs. Production systems agree: Cursor splits files "into semantic units — functions, classes, or ~500 token blocks [with] AST-based chunking [to] preserve code structure" [16], and function-level chunking is "the most common approach" in semantic code search because it is "specific enough to return useful results without being so granular that context is lost" [15].

Embed representations that learn from structure. A line of research builds embeddings explicitly designed to be robust to (or independent of) identifiers. NACS — "Deep Code Search with Naming-Agnostic Contrastive Multi-View Learning" — "strip[s] information bound to variable names from input ASTs and focus[es] on capturing intrinsic properties solely from ASTs' structures," then uses contrastive multi-view learning over graph and path views of the AST; when identifiers are perturbed, baseline accuracy drops "while the performance of NACS is not affected" [23]. The Canonical Identifier Naming (CIN) study takes the pragmatic version: rename all variables to canonical placeholders and strip comments, which both hardens models against lexical attacks and lets them operate on already-anonymized (decompiled-like) data; it notes GraphCodeBERT's use of data-flow as "a semantic-level structure of code" that improves understanding beyond syntax [26]. The code2vec studies close the loop empirically: training with obfuscated names yields a model that "more accurately reflects code semantics" and is "less reliant on variable names" [24]. The consistent message: **structural/data-flow features and naming-agnostic training are the way to make the embedding itself robust to missing identifiers** [23][24][26].

For genuinely identifier-free representations, borrow from binary code similarity detection (BCSD). BCSD is the discipline of finding semantically similar functions across compilers, optimizations, and obfuscation — i.e., with no reliable names at all — and it has produced embedding models that work purely from instructions and control flow. **Asm2Vec** learns function embeddings from CFG-sampled instruction sequences via a PV-DM (word2vec-style) model and is explicitly "robust ... against code obfuscation and compiler optimization" [28]. **jTrans** is "the first solution that embeds control flow information of binary code into Transformer-based language models," using a jump-aware representation; it outperforms prior SOTA on the diverse BinaryCorp dataset by 30.5% and doubles recall in real vulnerability search [27]. **BinShot** uses BERT-based transferable similarity learning with a weighted-distance metric to improve robustness on unseen, stripped functions [29]. These models won't directly answer a natural-language query, but they enable a powerful capability for messy code: **code-to-code semantic search** — given one function, find others that behave alike — and they offer architectural lessons (encode control flow explicitly; learn from structure; use contrastive/Siamese training) that transfer to higher-level decompiled code [27][28][29]. For Luau specifically, the analogous structural signals (the proto tree, instruction stream, upvalue/constant tables) are directly recoverable from bytecode with tools like Iridium [34], making structure-based similarity feasible even before high-quality decompilation.

Finding 5: Serve queries through hybrid retrieval with reranking, on a code-specialized embedding model

No matter how the index is built, the *retrieval architecture* that sits on top is where a large, reliable fraction of quality is won or lost — and for identifier-poor code it is not optional. The dominant, repeatedly validated pattern is hybrid search: run sparse lexical retrieval (BM25) and dense vector retrieval in parallel, fuse the ranked lists, then rerank the top candidates with a cross-encoder.

The reason hybrid search is essential here is that BM25 and dense retrieval "fail in orthogonal ways" [31]. Dense search captures conceptual similarity ("how do I authenticate a user?" matches auth code using different words), while "BM25 has the opposite profile: it matches exact tokens precisely, but cannot bridge semantic gaps" [32]. Dense-only retrieval "fails silently on exact terms, codes, and rare tokens" [31] — and decompiled code is full of exactly those: the few surviving signals are often literal constants, global/API names (Roblox : FireServer, game : GetService), error strings, and magic numbers that BM25 nails and a blurry embedding misses. Reported gains are substantial and larger for code: hybrid over dense-only "typically gives +10–25% MRR on technical corpora," and "on code search and technical documentation tasks, the improvement is larger (+10–15%) because exact identifier matching strongly benefits from BM25" [30][31][32].

The fusion mechanism of choice is **Reciprocal Rank Fusion (RRF)**, which "operates on ranks not scores, [solving] the score-incompatibility problem that makes naïve weighted averaging fail" [30]. The standard formula is $\text{Score}(d) = \sum 1/(k + \text{rank}_r(d))$, with $k = 60$ as "a reliable starting point" — lower it toward 30–40 to favor top-1 precision over deep recall [30][32]. After fusion, a **cross-encoder reranker** re-scores the top 50–100 candidates "with full query-document joint attention," because Recall@20 is not the product metric — only the top 3–5 chunks reach the LLM, and reranking pushes the right answer into that window [31]. The measured stack effect: dense-only → hybrid → hybrid+rerank yields "+20–35% MRR on realistic enterprise benchmarks" cumulatively [31]. The two-stage "bi-encoder retrieves, cross-encoder reranks" design is "the standard because it gets the best of both: the recall of fast approximate search, and the precision of deep relevance modeling" [31].

For the **embedding model**, code-specialized models clearly beat general text models on code retrieval. voyage-code-3 reportedly "outperforms OpenAI-v3-large and CodeSage-large by an average of 13.80% and 16.81%" across a large suite of code retrieval datasets, and supports Matryoshka dimensions plus int8/binary quantization to cut storage dramatically with minimal quality loss [9][10]. Other credible code/retrieval models named in the same comparisons include CodeSage, Jina-v2-code, CodeRankEmbed, and Nomic Embed Code [9]. One widely-circulated 2026 claim ranks a vendor model ("zembed-1") narrowly atop voyage on a code benchmark (0.6452 vs 0.6415 NDCG@10) [11]; this is a vendor blog with an obvious commercial incentive and a razor-thin margin, so it is reported only as a weak signal — the defensible conclusion is that several modern code-specialized embedding models are roughly comparable and all materially better than general-purpose text embeddings for code [9][11]. Two practical notes: (1) quantization (int8) reduces vector storage ~75% "with minimal impact on search quality," which matters for large decompiled corpora [9][17]; and (2) because summaries (Finding 3) are

natural language, a strong general or code model embeds them well — so when embedding *summaries* rather than raw code, model choice is less critical than when embedding raw decompiled text.

Finding 6: An end-to-end pipeline for a decompiled (Luau) codebase, and what production systems do

Synthesizing the findings yields a concrete, ordered pipeline. It is deliberately layered so that each stage degrades gracefully if a later stage is unavailable.

Stage 0 — Get the cleanest decompilation you can. Index quality is capped by input quality; recovery models "exhibit a strong dependency on the quality" of decompiler output [4]. For Luau, decompiler engineering directly recovers structure and even some names: SSA-based name preservation and pattern matching achieved "a 98.5% reduction in unnamed temporaries" in one Luau decompiler, by reconstructing if/while/for from jumps, collapsing temporaries, and preserving debug names where present [33]. Tools span the quality spectrum — AST-based decompilers like luauDec [35], bytecode deserializers like Iridium (which parses Luau versions 3–9, including Roblox MAC-envelope blobs, exposing protos, instructions, constants, and type info) [34], and hosted services like Sift [36]. Preserve every surviving signal: globals, constants, string literals, API calls, and the proto/call tree.

Stage 1 — Recover names and types (Finding 2). Run an LLM renaming/retyping pass over each decompiled function, propagating recovered names across the call graph in the LmPa style so that a named callee improves its callers [3][4]. This is language-agnostic and the most direct repair of the identifier deficit.

Stage 2 — Parse and chunk on structure (Finding 4). Build the AST (trivial for Luau given the decompiler already produces one [35]) and chunk via the cAST split-then-merge algorithm so functions stay intact and small siblings merge for density [12]. Attach metadata: file path, scope chain, call-graph neighbors, referenced globals/APIs [13].

Stage 3 — Summarize and enrich (Finding 3). Generate bottom-up NL summaries per function, then per file/module, seeded with repository-level context [20][21][22]. Prepend the structural metadata to each summary [13][20].

Stage 4 — Embed (Finding 5). Embed the enriched summaries (primary semantic index) and optionally the name-recovered code chunks (secondary), using a code-specialized model such as voyage-code-3 with int8 quantization for storage efficiency [9][17]. Keep raw decompiled code as the retrievable payload.

Stage 5 — Retrieve (Finding 5). Serve queries with hybrid BM25 + dense retrieval, fuse with RRF ($k \approx 60$), and rerank the top candidates with a cross-encoder before passing the top 3–5 to the consuming LLM [30][31][32]. Index incrementally — re-summarize and re-embed only changed units, tracked by content hashes [17].

This is not a theoretical construction; production code-search systems independently converge on its retrieval-side shape. Sourcegraph "combines three search paradigms ... keyword search ... structural search ... and semantic search," builds a Zoekt trigram index for fast lexical search, and ranks snippets with "BM25 and other learned signals" [15][18]; notably, Sourcegraph found embeddings operationally costly (sending code to an external embedder, storing/maintaining vectors) and replaced them for context retrieval with its native search engine for scalability [19] — a reminder that dense vectors are one tool, not the whole answer. Cursor splits code with AST-based chunking, tracks changes with a Merkle hash tree so "only modified files need re-uploading," embeds chunks with a custom code model trained on agent session traces, stores vectors with metadata in a vector DB, and combines semantic search with grep for precision — attributing roughly a 12% accuracy improvement to the semantic layer [16]. Turbo's reference design stores per-chunk name, signature, snippet, and line range; offers weighted BM25 (e.g., `name=5.0, signature=3.0`) and int8-quantized vector search; tracks file hashes for incremental indexing; and explicitly recommends combining "both approaches using Reciprocal Rank Fusion" [17]. Three production systems, three independent confirmations of: structural chunking, hybrid lexical+dense retrieval, metadata-rich chunks, and incremental indexing [15][16][17][18][19].

SYNTHESIS & INSIGHTS

Stepping back, the most important insight is that the decompiled-code problem and the general code-indexing problem are the *same problem viewed at different severities*. Every credible general-purpose code-search pipeline already hedges against identifier weakness — that is why hybrid search exists, why AST chunking prepends context, and why production systems combine semantic and lexical signals [13][15][16][17][30]. Decompiled code simply removes the safety margin: techniques that are "nice to have" on clean source become load-bearing. This reframes the user's fallback question — the "general best strategies" are not a separate Plan B; they are the same architecture, and the decompilation-specific techniques (name recovery, summarization-as-launders, structural embeddings) are *additions* that restore the signal those general strategies assume.

A second insight is that there are exactly three orthogonal ways to defeat the missing-identifier problem, and the best systems use all three rather than betting on one: **repair the input** (recover names/types — Finding 2), **replace the input with a surrogate** (NL summaries — Finding 3), and **represent what survives** (structure/control-flow/data-flow — Finding 4). These are complementary, not competing: name recovery makes both summaries and structural features sharper; summaries make natural-language queries land; structural features catch the cases where names and summaries are both unreliable. The convergence is striking — the reverse-engineering survey lists renaming, logic-restating, and summarization as the three dominant LLM uses for decompiled code [5], which is the same triad arrived at here from the retrieval side.

A third, non-obvious point concerns *where* the semantic burden should sit. The naive instinct is to find a magic embedding model that "understands" decompiled code. The evidence says the higher-leverage intervention is upstream: a cheap LLM summarization/renaming pass changes the data distribution from

"identifier-poor code" (where all embedding models struggle [24][25]) to "natural-language descriptions" (where all embedding models excel [20][21]). In other words, **invest in index-time enrichment before investing in a better embedding model** — the former addresses the root cause, the latter optimizes a constant factor.

Finally, the BCSD literature contributes a capability that pure text search overlooks: for decompiled code, *code-to-code* retrieval ("find functions like this one") is often more reliable than *text-to-code* retrieval, because structural similarity survives decompilation better than lexical similarity [27][28]. A complete system for a messy codebase should expose both query modes.

LIMITATIONS & CAVEATS

Several limitations bound these conclusions. First, **name-recovery accuracy is partial and domain-shifted**. The headline numbers — DIRTY 66.4% [1], VarBERT ~54% [2] — are for C/C++ from IDA/Ghidra; no source in this review directly benchmarks name recovery on Luau or game-script decompilation, so transfer is an informed extrapolation, not a measured result. The robust claim is the *technique* (LLM renaming + propagation), not a specific accuracy figure for Luau.

Second, **several retrieval-quality figures come from industry/practitioner sources rather than peer review**. The hybrid-search MRR gains [30][31][32], the voyage-code-3 comparisons [9][10], and the Cursor "12.5%" figure [16] are vendor or practitioner claims; they are mutually corroborating and directionally consistent with the academic chunking/retrieval literature [12][20], but exact magnitudes should be validated on the user's own corpus. The "zembed-1 is best" claim [11] is explicitly low-credibility (vendor blog, thin margin) and is not relied upon.

Third, **cost and freshness are real constraints not fully quantified here**. LLM summarization and renaming of an entire codebase is a non-trivial offline expense, and no source provided a clean cost-per-function benchmark for the full pipeline. Incremental indexing [16][17] mitigates ongoing cost but not the initial pass.

Fourth, **summaries can hallucinate**. Embedding a wrong summary indexes a function under the wrong meaning. Retaining raw code as the payload (so the consuming LLM verifies against ground truth) and keeping a lexical BM25 path are the practical guards, but summary quality remains a dependency on the summarizing model.

Fifth, **Luau-specific evidence is thin and largely practitioner-sourced** [33][34][35][36]; the decompiler-quality and tooling claims are credible engineering write-ups and repositories, not controlled studies. Finally, this review did not deeply evaluate graph-based retrieval (embedding the call/dependency graph itself) beyond its appearance in summarization seeding and BCSD; that is a plausible further axis not fully covered.

RECOMMENDATIONS

Immediate (build this pipeline). Implement the Stage 0–5 pipeline from Finding 6. Concretely: (1) maximize decompiler output quality first [33]; (2) run an LLM renaming/retyping pass with call-graph name propagation [3][4]; (3) AST-chunk at function granularity with metadata enrichment [12][13]; (4) generate hierarchical NL summaries and embed *those* as the primary index, keeping raw code as payload [20][21][22]; (5) serve via hybrid BM25+dense retrieval with RRF ($k \approx 60$) and a cross-encoder reranker [30][31][32]. This ordering front-loads the interventions with the best evidence-to-effort ratio.

Highest-leverage single step if resources are limited. Do summarize-then-embed (Finding 3). It is the cheapest way to convert the hard identifier-matching problem into the easy natural-language problem, and it is well supported [20][21][22]. Pair it with hybrid retrieval [30], which is low-effort and high-return.

Model choices. Use a code-specialized embedding model (e.g., voyage-code-3) with int8 quantization for the code/summary index [9][17]; use a capable but cost-efficient LLM for the batch summarization/rename passes; add a cross-encoder reranker on the top 50–100 candidates [31]. Always keep a BM25 lexical index over raw decompiled text to catch literals, API names, and constants that survive compilation [31][32].

Validate on your own corpus. Build a small labeled query set (50+ realistic queries with known answers) and measure Recall@5/MRR for dense-only vs. hybrid vs. hybrid+rerank, exactly as the practitioner guides recommend [31][32]. Vendor benchmark deltas do not transfer reliably; your corpus is the only benchmark that matters.

Expose code-to-code search. For a decompiled codebase, add a structural similarity capability (BCSD-style or AST/data-flow embeddings) so users can pivot from one function to behaviorally similar ones — a mode where structure outperforms text [27][28][23].

Further research / next steps. If accuracy is insufficient: fine-tune or adapt a naming-agnostic encoder (NACS/CIN style) on your own decompiled corpus [23][26]; experiment with graph-aware retrieval over the call/dependency graph; and consider an agentic retrieval loop (as Sourcegraph Deep Search and Cursor use) that iteratively searches and reads rather than relying on a single nearest-neighbor lookup [15][16].

METHODOLOGY APPENDIX

Mode and process. This was a deep-mode deep-research run executed on 2026-06-21. The pipeline followed the eight-phase methodology: scope and plan; parallel retrieval; triangulation; evidence-driven outline refinement; synthesis; critique; refinement; and packaging. Retrieval was conducted via parallel web search across eight decomposed angles: (1) decompiled-code variable-name recovery; (2) code embedding models; (3) summarize-then-embed; (4) AST/repository chunking; (5) LLM-based decompilation understanding; (6) binary code similarity / stripped-binary search; (7) hybrid retrieval and

reranking; and (8) identifier-robust / naming-agnostic representations, plus targeted searches on production code-search systems and Roblox/Luau decompilation.

Sources and triangulation. 36 sources were registered with stable content-hash IDs in `sources.jsonl`; display numbers were assigned at render time. The mix spans peer-reviewed venues (USENIX Security, IEEE S&P, ISSTA, ACM CCS, EMNLP, ACSAC), arXiv preprints, vendor engineering documentation (MongoDB/Voyage, Sourcegraph, Turbo, Cursor coverage), and practitioner write-ups/repositories (Luau tooling, hybrid-search guides). Major claims are supported by 3+ independent sources where possible: the identifier-dependence claim by [23][24][25] + [4][5]; name recovery by [1][2][3][4][6]; summarize-then-embed by [20][21][22] + [13]; AST chunking by [12][13][15][16]; hybrid retrieval by [30][31][32] + [17]; production convergence by [15][16][17][18][19].

Credibility handling. Peer-reviewed sources were prioritized for mechanism claims; industry/practitioner sources were used for quantitative retrieval-gain figures and current model comparisons, and are flagged as such in Limitations. One vendor leaderboard claim [11] was explicitly down-weighted. No citations were fabricated; where authorship metadata was not captured during retrieval, entries are marked generically rather than invented.

Assumptions adaptation. The original plan weighted decompilation-specific techniques and general fallback strategies roughly equally. Evidence refinement (Phase 4.5) revealed that the two are better framed as one layered architecture rather than a primary/fallback split, and that summarize-then-embed deserved promotion to a standalone primary finding given its strong, convergent support. The outline was adjusted accordingly while retaining the original question's core.

Reproducibility. Run configuration is in `run_manifest.json`; the source registry is `sources.jsonl`. Full-text captures of fetched pages were stored to the agent tool cache during retrieval and used for quotation verification.

BIBLIOGRAPHY

- [1] Chen, Q., Lacomis, J., Schwartz, E. J., Le Goues, C., Neubig, G., & Vasilescu, B. (2022). "Augmenting Decompiler Output with Learned Variable Names and Types (DIRTY)". USENIX Security 2022. <https://arxiv.org/abs/2108.06363> [2] Pal, K. K., Bajaj, A. P., et al. (2024). "'Len or index or count, anything but v1': Predicting Variable Names in Decompilation Output with Transfer Learning (VarBERT)". IEEE S&P 2024. https://www.atipriya.com/files/papers/varbert_oakland24.pdf [3] Xu, X., Zhang, Z., Feng, S., Ye, Y., Su, Z., et al. (2023). "LmPa: Improving Decompilation by Synergy of Large Language Model and Program Analysis". arXiv. <https://arxiv.org/abs/2306.02546> [4] Xie, D., Zhang, Z., Jiang, N., Xu, X., Tan, L., & Zhang, X. (2024). "ReSym: Harnessing LLMs to Recover Variable and Data

Structure Symbols from Stripped Binaries". ACM CCS 2024.

<https://www.cs.purdue.edu/homes/lintan/publications/resym-ccs24.pdf> [5] Authors of the SoK (2025). "SoK: Potentials and Challenges of Large Language Models for Reverse Engineering". arXiv. <https://arxiv.org/pdf/2509.21821> [6] Authors of REBench (2026). "REBench: A Procedural, Fair-by-Construction Benchmark for LLMs on Stripped-Binary Types and Names". arXiv. <https://arxiv.org/pdf/2604.27319> [7] Tan, H., Luo, Q., Li, J., & Zhang, Y. (2024). "LLM4Decompile: Decompiling Binary Code with Large Language Models". EMNLP 2024. <https://aclanthology.org/2024.emnlp-main.203/> [8] Authors (2026). "Large Language Models for Binary Decompilation: Paradigms, Evaluation, and Challenges". CNSSE 2026 (ACM). <https://dl.acm.org/doi/10.1145/3803633.3803666> [9] Voyage AI / MongoDB (2024). "voyage-code-3: More Accurate Code Retrieval With Lower Dimensional, Quantized Embeddings". MongoDB Blog. <https://www.mongodb.com/company/blog/voyage-code-3-more-accurate-code-retrieval-lower-dimensional-quantized-embeddings> [10] MarkTechPost (2024). "Voyage AI Introduces voyage-code-3: Embedding Model Optimized for Code Retrieval". <https://www.marktechpost.com/2024/12/09/voyage-ai-introduces-voyage-code-3-a-new-next-generation-embedding-model-optimized-for-code-retrieval/> [11] ZeroEntropy (2026). "The Best Embedding Model for Code in 2026: zembed-1 Tops the Leaderboard". ZeroEntropy Blog (vendor; low-credibility). <https://zeroentropy.dev/articles/best-embedding-for-code/> [12] Authors (2025). "cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree". arXiv. <https://arxiv.org/html/2506.15655v1> [13] Supermemory (2025). "Building code-chunk: AST Aware Code Chunking". <https://supermemory.ai/blog/building-code-chunk-ast-aware-code-chunking/> [14] LanceDB (2024). "Building RAG on codebases: Part 1". <https://www.lancedb.com/blog/building-rag-on-codebases-part-1> [15] Sourcegraph (2025). "Semantic Code Search: What it is and how it works". <https://sourcegraph.com/blog/semantic-code-search-what-it-is-and-how-it-works> [16] DigitalApplied (2025). "Cursor AI Semantic Search: 12.5% Better Code Agent Accuracy". <https://www.digitalapplied.com/blog/cursor-semantic-search-coding-ai-guide> [17] Turso (2025). "Code Indexing with Turso (FTS + vector hybrid)". <https://docs.turso.tech/guides/code-indexing> [18] Sourcegraph (2025). "Sourcegraph architecture (Zoekt trigram index)". <https://sourcegraph.com/docs/admin/architecture> [19] Sourcegraph (2024). "How Cody understands your codebase". <https://sourcegraph.com/blog/how-cody-understands-your-codebase> [20] Authors (2025). "Natural Language Summarization Enables Multi-Repository Bug Localization by LLMs in Microservice Architectures". arXiv. <https://arxiv.org/pdf/2512.05908> [21] Authors (2025). "RepoSummary: Feature-Oriented Summarization and Documentation Generation for Code Repositories". arXiv. <https://arxiv.org/html/2510.11039> [22] Authors (2025). "Hierarchical Repository-Level Code Summarization for Business Applications Using Local LLMs". arXiv. <https://arxiv.org/html/2501.07857> [23] Authors (2024). "Deep Code Search with Naming-Agnostic Contrastive Multi-View Learning (NACS)". arXiv. <https://arxiv.org/html/2408.09345v2> [24] University of Waikato (2022). "Obfuscating Variable Names to Improve code2vec Robustness". <https://researchcommons.waikato.ac.nz/server/api/core/bitstreams/cd81090b-79fe-44a4-bf0a-7fb17b0a8111/content> [25] Authors (2025). "Evaluating Variable Name Anonymisation for Code2Vec Robustness". HAL. <https://hal.science/hal-05461814v1/document> [26] ETH Zurich (2022). "Canonical Identifier Naming on Code Search Models". <https://pub.tik.ee.ethz.ch/students/2022-HS/GA-2022-07.pdf> [27] Wang, H., Qu, W., Katz, G., Zhu, W., Gao, Z., Qiu, H., Zhuge, J., & Zhang, C. (2022). "jTrans: Jump-Aware Transformer for

Binary Code Similarity Detection". ISSTA 2022.

<https://dl.acm.org/doi/10.1145/3533767.3534367> [28] Ding, S. H. H., Fung, B. C. M., & Charland, P. (2019). "Asm2Vec ... Robustness against Code Obfuscation and Compiler Optimization" (with BCSD survey context). IEEE S&P 2019.

<https://tyeyeah.github.io/2022/03/26/2022-03-26-Binary-Code-Similarity-Detection-Papers/>

[29] Ahn, S., Ahn, S., Koo, H., & Paek, Y. (2022). "Practical Binary Code Similarity Detection with BERT-based Transferable Similarity Learning (BinShot)". ACSAC 2022.

https://www.acsac.org/2022/program/papers/76-Ahn-Software_Security_I.pdf [30]

DigitalApplied (2026). "Hybrid Search: BM25, Vector & Reranking Reference 2026".

<https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026> [31]

Tensoria (2026). "Dense RAG Fails on Rare Terms. Hybrid Search Fixes It [2026 Guide]".

<https://tensoria.fr/en/blog/hybrid-search-reranking> [32] atomic-rag (2025). "Hybrid Search

(BM25 + dense + cross-encoder) technique notes". [https://github.com/rohinp/atomic-](https://github.com/rohinp/atomic-rag/blob/main/docs/techniques/hybrid-search.md)

[rag/blob/main/docs/techniques/hybrid-search.md](https://github.com/rohinp/atomic-rag/blob/main/docs/techniques/hybrid-search.md) [33] Brandwijk, K. (2025). "From 10,000 to

18: Drastically Improving Luau Bytecode Decompilation Quality". DEV Community.

[https://dev.to/kbrandwijk/from-10000-to-18-drastically-improving-luau-bytecode-](https://dev.to/kbrandwijk/from-10000-to-18-drastically-improving-luau-bytecode-decompilation-quality-2l6p)

[decompilation-quality-2l6p](https://dev.to/kbrandwijk/from-10000-to-18-drastically-improving-luau-bytecode-decompilation-quality-2l6p) [34] ActualMasterOogway (2025). "Iridium: Luau bytecode

deserializer and disassembler". GitHub. <https://github.com/ActualMasterOogway/Iridium> [35]

xgladius (2024). "luauDec: AST-based decompiler for Luau". GitHub.

<https://github.com/xgladius/luauDec> [36] Sift (2026). "Sift: readable Luau from Roblox bytecode (decompiler service)". <https://siftrblx.com/>